Pi-Piper

Peyton Hollenback

CST-451 Capstone Project Proposal

Grand Canyon University

Instructor: Professor Mark Reha

Revision: 1.0

Date: 09/22/24

© 2020. Grand Canyon University. All Rights Reserved.

ABSTRACT

Republic Pi's current order management system involves manually comparing prices between Sysco and US Foods using a paper prep list and an iPad. To streamline this process, the proposed app will fully digitize and automate the restaurant's prep list and order management system. On the admin side of the app it will compare prices between suppliers, generate the most cost-effective orders, and offer suggestions based on past data and current needs. Also exclusively on the admin side will feature a prep list and ingredient CRUD tool. Additionally, the app will feature a visualized prep list and planning tool to assist employees in pre-open and pre-rush preparation. With presets and automated suggestions, the app reduces the need for manual input, optimizing inventory restocking and prep processes. The aim is to create a lightweight, dynamic admin system that saves time, increases productivity, and requires minimal oversight, ensuring that managers can focus on other critical aspects of running the restaurant.

History and Signoff Sheet

Change Record

Date	Author	Revision Notes
09/22	P. Hollenback	Initial draft for review/discussion

Overall Instructor Feedback/Comments

Overall Instructor Feedback/Comments

Integrated Instructor Feedback into Project Documentation

☐ Yes ☐ No

Project Approval

☐ Professor Mark Reha

TABLE OF CONTENTS

Project Overview and Project Objectives

The problem:

Since its conception Republic Pi has used a paper prep list and a company ipad to make its orders between the two top services in the area, Sysco and US Foods. In the past, the manager making the orders has had to juggle multiple devices in order to compare prices to make the most effective order per the restaurant's needs. Also, the prep list and pre-service plan has been on paper since the restaurant's first day, this setup limits flexibility and overall efficiency of the restaurant. The goal is to digitize the prep list and ordering process in order to streamline operations, improve price comparison, and enable managers to make necessary adjustments whenever necessary with ease.

Project Objectives

- ☐ Create a mutable, easy to understand prep list, that is categorized adequately
- ☐ Create a login system for admin and otherwise that will authenticate user abilities
- ☐ Visualize the prep plan and pre rush plan
- ☐ Facilitate price comparison
- ☐ Develop a low maintenance service
- ☐ Enable expansion to other restaurants
- ☐ Host on site to minimize costs
- ☐ Create a future proof application

Challenges

1. App scraping from Sysco foods
2. App scraping from US foods
3. Suggestion system based on ordering history, and other contextual information.
4. Keeping the ability to host and maintain the app with low cost.
5. Making an app that allows manual insertion of other stores and their needs.
6. Allowing a manager to add a restaurant, log into the restaurant and have the correct state
7. Maintaining flexibility while leaving the door open for future integrations of features(especially based around ML)

Benefits and Opportunities

The restaurant will benefit from increased efficiency across all processes in the back of house by streamlining prep list management and ordering functions. Another benefit is the flexibility of the tools included in the app, by allowing a mutable prep list and potential for new store insertion there is lots of room for the company to add the app into the general workflow of all of its operations. By collecting data en masse about the day to day operations of the restaurants, the potential business insights are endless, with lots of possibilities for collecting more and using these numbers to enhance company efficiency. The idea of expanding to multiple restaurants will also increase the communication and teamwork between the restaurants, making the management of all three far easier with less overhead required.

Project Scope

1. **The following states the project scope, as well as the items that may be borderline or out of scope:**

   The items that are definitively in scope include prep list management for each day, admin price comparison across multiple provider companies(automated or mock), pre-prep and pre-rush planning, admin side prep and ingredient list management, and a recipe book digitized. The borderline items include a suggestion system once hooked up to order history, the ingredients that are a part of all prep items etc., as well as app scraping for both markets. For now, the items completely out of scope include user management, which would include signing in users and logging their weekly hours. As well as expanding the system to the front of house as well that would include ordering their items and managing double the users.

2. **Project stakeholders:**

Stakeholder Name	Role(s)	Responsibilities
Darrin Gleason	Restaurant owner, back of house manager	Defining needs of the app and determining what is necessary for scope.
Peyton Hollenback	App developer and point man.	Developing said app.

3. **The work breakdown structure outlines all that will need to be completed in order for this project to be completed:**

Work Breakdown Structure										
ID	Task	Dependencies	Status	Effort Hours	Cost	Start Date	Planned Completion	Estimate to Completion	Actual Completion	Resource
1	Complete project proposal	Defining scope and abilities of the app	Completed	3	N/A	09/06	09/22	2 weeks		Research
2	Write Functional User Stories and Use cases	Complete user stories for all features in scope	Not complete	2	N/A	09/23	09/25	2 days		Requirements
3	Write Non-Functional User stories and Use Cases	Complete non functional table	Not Complete	2	N/A	09/25	09/27	2 days		Requirements
4	Outline Tools and Technologies	Complete tools and	Not Complete	4	N/A	09/27	10/01	5 days		Research

		technologies table								
5	Create Logical System Design	Make diagram of logical system design	Not Complete	2	N/A	10/01	10/02	1 Day		Drawing
6	Draw User Interface Diagrams	Plan and draw out each UI diagram	Not Complete	5	N/A	10/02	10/05	3 days		Drawing
7	Create Security Matrix/DB design	Outline who is allowed to do what	Not complete	8	N/A	10/05	10/08	3		Report
8	Create Reports Design	Create report for any necessary element	Not Complete	3	N/A	10/08	10/11	3 days		Report
9	Summarize planning phase/outline final requirements	Completion of previous steps	Not complete	3	N/A	10/13	10/15	2 days		Report
10	Create overview of project solution	Complete all requirements and technology definitions	Not complete	3	N/A	10/15	10/16	1 day		Report
11	Review details of requirements	Review and revise final requirements	Not complete	8	N/A	10/16	11/01	2 weeks		Revision
12	Collect pertinent business data	Begin gathering data	Not complete	5	N/A	11/01	11/05			Coding/Designing
13	Confirm proof of concepts	Write or revise vertical functionality	Not complete	8	N/A	11/05	11/12	1 week		Coding
14	Plan user stories for coming weeks and second semester	User stories table	Not complete	5	N/A	11/12	11/15	3 days		Planning

15	Ensure all ready for development and testing phase	User stories plan	Not complete	1	N/A	11/16	11/16	1 day		Planning
16	Create test cases	Project outline	Not complete	10	N/A	11/17	11/20	1 week		Planning
17	Begin development first phases(Project structure and simple functionality)	Functional design	Not complete	7	N/A	11/20	11/27	1 week		Coding
18	Develop backend structure/effectiveness	Functional design	Not complete	10	N/A	11/27	12/12	2 weeks		Coding
19	Develop data processing unit	Functional design	Not complete	15	N/A	12/12	01/01	3 weeks		Coding
20	Develop UI of prep list/home	UI diagrams	Not complete	10	N/A	01/02	01/16	2 weeks		Coding
21	Develop UI of ordering functionality	UI diagrams	Not complete	10	N/A	01/17	01/30	2 weeks		Coding
22	Develop Admin UI for ingredient and prep management	UI diagrams	Not complete	10	N/A	02/01	02/			

Project Success Measures

1. **The project will be complete once these items are fully implemented:**

Project Completion Criteria
1 - Implement the app into the main restaurant
2 - Admin authentication
3 - Digitized real time Prep list management
4 - Price comparison and easy order management
5 - Suggestion system based on order history and prep list completion
6 - On site hosting and low maintenance
7 - Allow for prep list management offline using PWA

2. **The table shows what can be assumed about the project and what initial constraints I suspect could affect the systems effectiveness:**

Assumptions and Constraints					
ID	Description	Comments	Type	Status	Date Entered
1	I can assume the prep list will be updated frequently.	This is typically done each night. Because of this there needs to be well maintained data to not mix up the days.	Data	True	09/21
2	I can assume the manager only needs to order from the food markets twice a week.	This will go into play when deciding how often the python script is, and when dealing with data freshness whenever the owner decides to use the tool	Exterior conditions	Open	09/21
3	I can assume the restaurant has a reliable network connection.	The company uses plenty of technology to keep the company up and running already.	Network	True	09/21
4	I can assume that the Wintel machine hosting the app will not be disturbed for any reason.	I plan on providing a machine that will be identified as the sole component hosting the app.	Hosting	True	09/21
5	I can assume there will be limited concurrent users.	On a daily basis there will be 3-4 users at most. This limits any need for asynchronous processing.	Code standards	True	09/21
6	I can assume the rate of data growth will not affect the system.	Although the data logging happens daily, this should not affect the app's performance for a long time.	Data management	True	09/21
7	A constraint may be revealed if the system expands to multiple restaurants.	Data growth, concurrent users, and overall overhead of the app would grow significantly.	Performance	Open	09/21
8	A constraint is the data backup that would otherwise take place if the app were in the cloud.	There is a lot of risk by keeping the disaster recovery and backup manually	Data integrity	Open	09/21

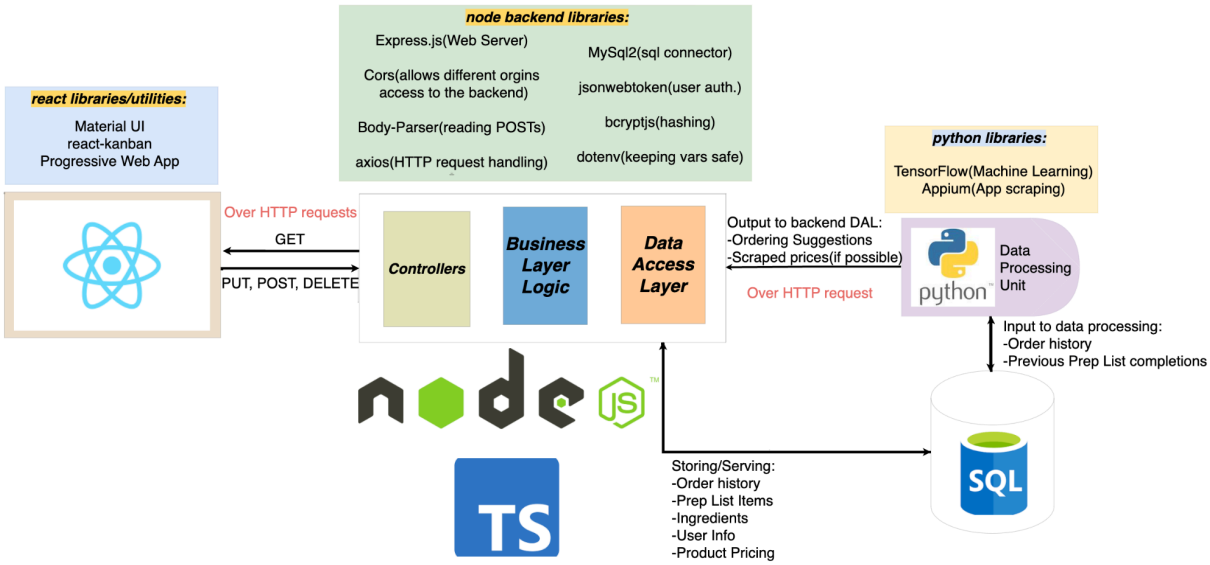
Project High-Level Solution

Introduction

Pi-Piper will use the daily logging of prep tasks alongside restaurant order history in order to generate a framework for the company to use when completing admin-like tasks such as stocking inventory, and completing and updating prep management. The app will be web based, opening it up to expanding future infrastructure. will be able to be accessed from the ipad home screen or any related mobile device. An option to log in upon the app opening will provide context to the app which will allow authenticated managers to access the ordering capabilities. When completing ordering tasks, there will be up to date pricing on all ingredients provided by the specific restaurant's needs. For now, this data will be mock until determining if the app scraping from the companies can be fulfilled. When it comes to the prep list, I plan to create a kanban board that will identify “To-Do”, “In-progress”, and “Completed” items. The prep list can be determined each night by the manager after close to figure the items that will be completed the following morning. Before morning prep each day is when the list will be updated for the crew.

Solution

The app will be developed using React with PWA, this will allow the app to be represented as a mobile application that also has plenty of offline functionality, ensuring it is always readily available due to the caching that PWA handles once a device accesses the application. Strong typing will be a major part of my overall solution. Data integrity is of utmost importance in this project due to the business concerns that arise from having a poorly populated prep list or order suggestions, or even a fatal error that may come from data that is missing type specific data. Typescript will be used in both the front and backend of my project, just to make sure the objects will be ensuring all data is correct at any given time. I will be using React for my frontend, node js for my backend, python for the scraping and ML suggestion creation, and SQL as the sole datasource:



I plan to log more information than what is initially necessary to my SQL datasource. Should this app expand in the future, the logging of data daily will prove beneficial, especially with a Machine Learning aspect at the heart of it all; in what is known as “data-driven design”, there is little downside to creating a datasource with attributes that may not be used now, but if logged for long enough there would be plenty of use cases for it in the future. For now, users will be a basic table, but the ingredient price logging, and prep list management logging should be robust. Including prices for all market sources on all particular ingredients will only help the machine learning components and be ignored in all other areas, not affecting the overall performance of the app. Prep list management could even be logged down to start and end times, identifying how long it should take to complete each task, which would over time improve the visualization of the pre-open and pre-rush processes. The python data processing unit will require a Machine Learning library capable of expanding and potentially handling larger data sets over time. TensorFlow is the library that I have decided on for that process, which over time should programmatically determine factors that will improve suggestions. In theory, it will require hosting the suggestion data once each day before employees come in for the day over an HTTPS request. This request needs to be limited to once a day, because the price comparison unit could implement price scraping in the future, which should be limited in order to not be flagged by the provider. The backend requires gathering

data from the python data processing unit and the sql tables in order to complete all business logic. The business logic will include managing the manager's cart for ordering and comparing the provided pricing(whether mock or otherwise), maintaining the state of the prep list, and ensuring data freshness by checking for updates whenever necessary. Admin sign in and authentication as well as data security will all be implemented as well, by utilizing JSON Web Tokens, and ensuring all sensitive data is properly hashed before being posted to the datasource. Ideally, this will only need to be done for the managers to sign in and the rest of the employees who use the app will go through the default gateway that only allows all prep list completion and planning visualization. Upon the app being launched, this is the proposed program flow of an admin user:

1. Admin uses code at login screen(JWT auth.)
2. Fetch ingredient data and prep list from SQL
3. Fetch order suggestions from data processing unit
4. Fetch pricing on all current ingredients
5. User has ability to add/update/delete prep list or ingredient list
    a. Updates sent back to SQL database
6. User has ability to begin price comparison and cart management
7. User can complete all non-admin tasks

For a non-admin user upon app launch:

1. Fetch all prep list data from SQL
2. User marks tasks as completed or in progress
    a. Task status updates sent to SQL database
3. Track prep completion progress

When it comes to the prep list and ingredient CRUD tool, there are steps that will be put into place to ensure the security of the managers business elements. In order to ensure the integrity of the owners/managers machine and products, there will be no connection when it comes to updating data; all updates to prep list or new ingredients will be handled in an admin side management page. If a new item

is ever added to the menu, this can be handled through the app rather than using an excel datasource on the owners machine or a developer having to update the database manually. Incorporating these technologies and strategies ensures this app will be secure and adaptable for the future needs of the business. Starting at the app's launch, allowing offline functionality and reliable page caching, there will be limited downtime regardless of network connectivity, while also ensuring data integrity at all times by developing with typescript. By always incorporating extensive data logging it will fuel the python data processing unit over time for increasingly better results. Data freshness and mutability on a day to day basis for things like prep list items, ingredients, and prep plan visualization all will be done through the data flow up from sql through the python data processing unit and ultimately served by my node.js backend. Admin users will have access to secure management tools for data updates, while non-admin users will interact with prep lists and planning through the app, ensuring a robust and scalable solution to business issues.

Project Controls

1. **This table outlines the potential risks that I feel I may encounter when creating this application, as well as the overall risk impact and my contingency plan if it did occur:**

Risk Management				
Event Risk	**Risk Probability (high, medium, low)**	**Risk Impact**	**Risk Mitigation**	**Contingency Plan**
What is the risk?	What is the probability?	What is the impact if the risk occurs?	What can be done to minimize the risk?	What can be done to minimize the impact of the risk?
Hardware failure of the Wintel machine	Low	Can crash the entire system and leave the staff to pick up the pieces, if they are completely entrenched in the software this is fatal.	*Perform regular maintenance checks * Backup data regularly	If a failure occurs, switch to a backup machine on premises to try and limit downtime.
Fatal software bugs	Medium	Bugs in the app could cause data inconsistencies, crashes, or brief disruption in services.	* Conduct thorough testing, and make sure unit tests check for all edge cases or otherwise *Use GIT for version control in order to rollback versions when necessary	If a bug creates downtime, quickly alert developers in order to rollback the changes or fix the issue in as short a time as possible.
Network Failure	Low	Would impact order management and prep list completion	* PWA limits the effects of offline networks by caching the important aspects of the app	When setting up PWA, clearly define what aspects should be cached when a device first accesses the app when it has network connection

Scraping Fails	Medium	If scraping is implemented, being blocked from a site for over usage could completely fold the tool.	* Limit price checks to once a day before the order takes place,	Scraping prices frequently could flag the app and get my service blocked from further scraping, so only scraping upon request or twice a week.
Non-admin accessing ordering functionality	Low	This would potentially allow non-admin to use the funds of the company for unauthorized purchases.	* Provide multiple authentication methods throughout the process(upon purchase, upon sign-in)	Ensure there are authentication methods in place, for app users and even hackers that could use the api.

2. The issues log outlines the issues that I feel I should address at the beginning of development, as well as the expected issues that I will need to confront:

Issues Log							
ID	**Description**	**Project Impact**	**Action Plan/Resolution**	**Importance**	**Date Entered**	**Date to Review**	**Date Resolved**
1	What is the issue?	How will this impact scope, schedule & cost?	How do you intend to deal with this issue?				
2	Running an external python script that is triggered by the node backend.	If I get to the point where this script affects the performance significantly, I may need to change my tech stack around considerably.	Keep the python script as thin as possible, and modularize the scraping and suggestion system, so it does not happen all at once, only upon request of each module.	HIGH	09/21		
3	Will a Wintel computer be able to manage the potential expansion to 3 restaurants?	The price skyrockets if I get to the point where I want to expand and immediately have to put more funds into hosting and maintaining my system.	Manage storage as best as I can, utilizing external storage to ensure the main machine is working on the necessary processing first.	MEDIUM	09/21		

3. The change control log lists the changes that could be put into my system over time, whether expected or not:

Change Control Log							
ID	**Change Description**	**Priority**	**Originator**	**Date Entered**	**Status**	**Date of Decision**	**Included in Rev. #**
1	Transferring data of pricing for ingredients from mock to scraping	LOW	Developer	09/22	No change yet	?	?
2	There may be changes in design of UI or data collection depending on user request.	LOW	Developer	09/22	No change yet	?	?

Project Cost and Schedule

1. **Costs: (NOTE: there is on-site network connection, electricity)**

id	Item	Comment	Cost
1	Wintel computer	Assume i am aiming for Intel Core i5, 16GB ram, with 512 GB running windows)	Between 500-1000$
2	External harddrive	For data backup and disaster recovery	50$
		TOTAL:	Approximately 600$

2. **The following are the project phases and tasks for each phase broken down across the coming months for planning, and towards the end of December and into the new year for implementation and testing:**

PHASE 1: Requirements and Analysis

1. Complete project proposal by defining the scope and abilities of the app(this will be marked completed upon instructor feedback and implementation.
    - Duration: 2 weeks 9/6-9/22
    - Priority: HIGH, must be complete and thorough for the rest of the project to go smoothly

2. Write functional User stories and use cases - complete for in scope requirements
    - Duration: 2 days 9/23-9/25
    - Priority: HIGH, dictates the functionality and user experience as well as developer schedule

3. Write non-functional user stories and use cases
    - Duration: 2 days 9/25-9/27
    - Priority: MEDIUM, should still be met, but don't hold as high of priority as functional. More so written for the appeasement of stakeholder requirements and user experience.

4. Fully outline Tools and Technologies, and Frameworks that will be used
    - Duration: 4 days 9/27-10/01
    - Priority: MEDIUM, will allow easy development if there are well defined technology usage.

5. Create Logical System design(diagram each units modules and relation)
    - Duration: 1 day 10/01-10/02
    - Priority: MEDIUM, does not affect whether or not completion is possible, more so for clarity.

6. Draw User Interface Diagrams
    - Duration: 3 days 10/02-10/05
    - Priority: HIGH, these drawings are what trigger almost all functionality

7. Create security matrix/database design(who is allowed in what system and ER)
    - Duration: 3 days 10/05-10/08
    - Priority: HIGH, effective data management will be crucial in order to create a effective backend with little bloat

8. Create Reports design for necessary elements(ordering unit, task management)
    - Duration: 3 days 10/08-10/11
    - Priority: MEDIUM, crucial for program effectiveness, but not necessarily overall functionality

9. Summarize planning phase and outline final requirements
    - Duration: 2 days 10/11-10/13
    - Priority: MEDIUM, ensuring all details are clear and concise, and everything coincides with scope

10. Create overview of project solution
    - Duration: 1 day 10/15-10/16
    - Priority: MEDIUM: more so for clarity

11. Review/Revise details of requirements
    - Duration: 2 days 10/16-11/01
    - Priority: HIGH, redefine necessary details and potentially scope

Phase 2: Development Preparation

1. Collect pertinent business data
    - Duration: 5 days 11/01-11/05
    - Priority: HIGH, this is a data-driven application

2. Confirm Proof of concepts
    - Duration: 3 days 11/05-11/12
    - Priority: HIGH, needs to be known if the concepts are possible to begin with

3. Plan User Stories for Coming weeks

- Duration: 3 days 11/12-11/15
- Priority: MEDIUM, necessary for developer planning

4. Ensure readiness for development and testing phase
    - Duration: 1 day 11/16
    - Priority: HIGH, all things must be in order before beginning development

Phase 3: Development

1. Database setup and Backend development
    - Tasks: setup SQL database, create endpoints and establish data flow
    - Duration: 2 weeks 11/17-12/05

2. User authentication and security
    - Tasks: Implement JWT authentication, user roles and implement sensitive data protection in the code base
    - Duration: 1 week 12/06-12/13

3. Develop core frontend components
    - Task: Create login, dashboard, and inventory management screens
    - Duration: 1.5 weeks 12/14-12/24

4. Develop secondary frontend components
    - Task: Create Kanban prep list screen, pre-prep and pre-rush screens, and price comparison screens
    - Duration: 2 weeks 1/02-1/15

5. Inventory management and order tracking
    - Finalize order interface and integrate with the backend, logging order history
    - Duration: 3 weeks 1/16-2/07

6. Data processing unit development
    - Tasks: Develop the backend module for tracking relevant data, and developing data analytics tools
    - Duration: 2 weeks 2/08-2/22

Phase 4: Testing and Deployment

1. Unit testing
    - Tasks: complete unit tests on backend API's, frontend components, and data processing unit
    - Duration: 2 weeks 2/23-3/03

2. Integration Testing
    - Tasks: Ensure smooth app operation and data flow
    - Duration: 1.5 weeks 3/04-3/13

3. Use acceptance testing
    - Tasks: Introduce to staff and managers and test real world usage
    - Duration 1.5 weeks 3/14-3/23

4. Final Deployment
    - Tasks: Deploy the application to the Wintel machine in the restaurant and ensure app runs smoothly
    - Duration: 1 week 3/24-3/29

Appendix A – References

None.

Appendix B – Copyright Compliance

React: licensed under MIT, allowing free use modification and distribution.

Node.js: licensed under MIT.

React-Kanban, JWT, MUI, Express.js, Cors, Body-Parser, Axios: licensed under MIT.

PWA: utilizes workbox and service workers, which are both licensed under MIT

TensorFlow, Appium: licensed under Apache License 2.0, which is a permissive open source license.